Software-Hardware Co-design for Data-Intensive Systems

Lecture 6: Software-Hardware Co-Design Methodology

辛艾曼 Aman Sinha

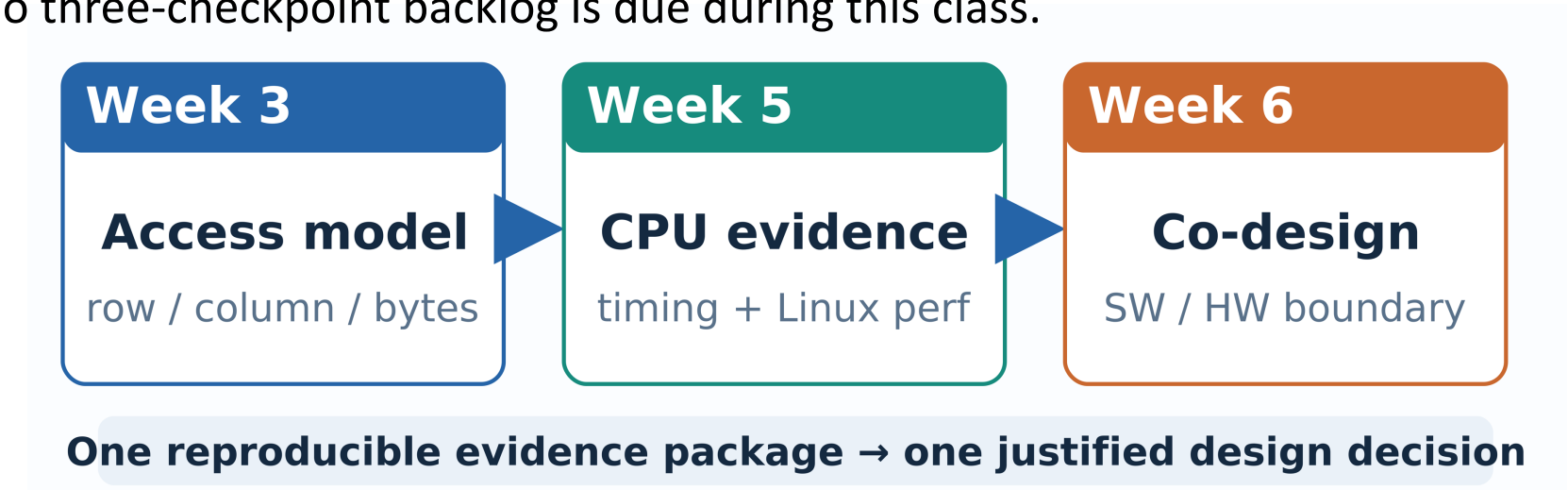
Institute of Integrated Circuit Design

National Sun Yat-Sen University

國立中山大學
National Sun Yat-sen University

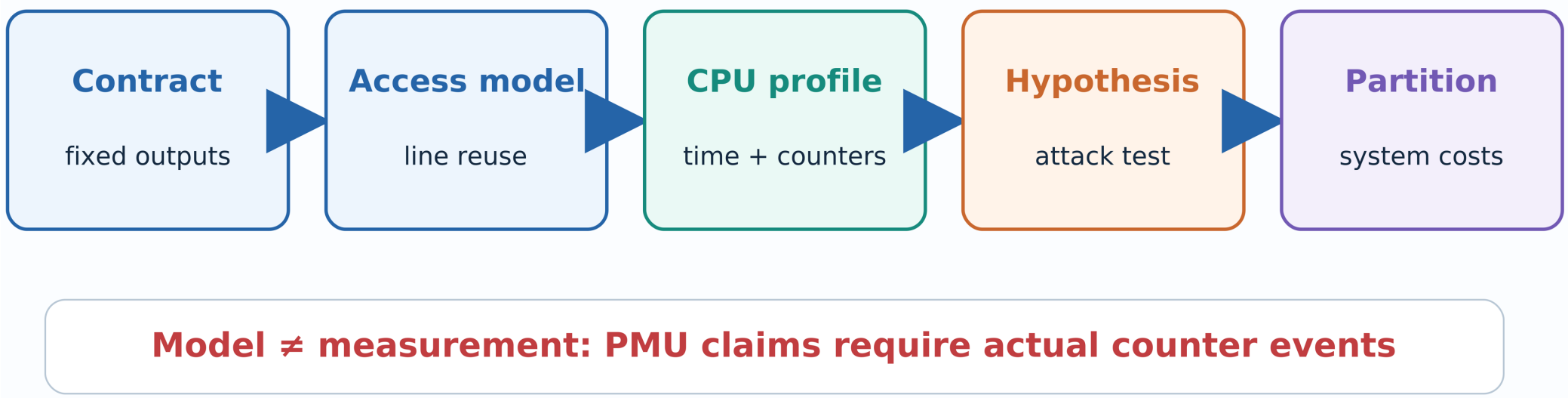
Week 6 — Integrated Guided Lab

- **Recover:** Week 3 workload and memory-access analysis.
- **Measure:** Week 5 baseline and bottleneck evidence with Linux perf.
- **Design:** Week 6 software–hardware partition and verification plan.
- **Case study:** Compiled C++ CSR sparse matrix–vector multiplication.
- **Format:** Three 50-minute sessions; one integrated evidence package.
- **Fairness:** No three-checkpoint backlog is due during this class.



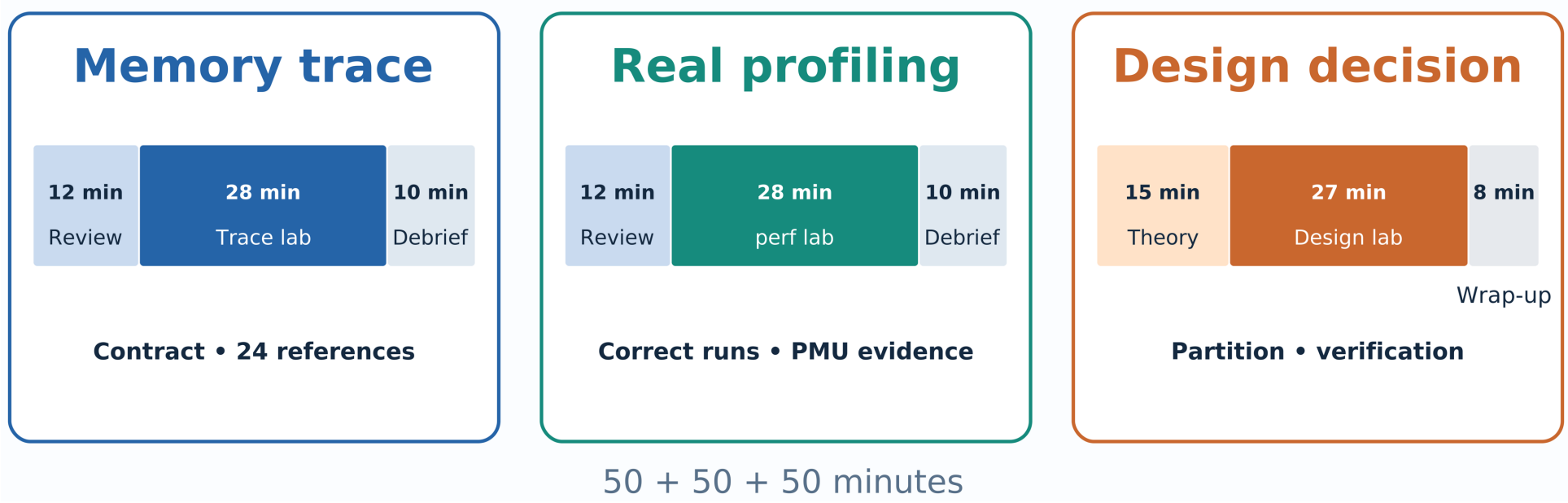
Learning Objectives and Evidence

- **Specify:** Fix inputs, output oracle, work unit, and timed boundary.
- **Analyze:** Predict memory footprints, reuse, and cache conflicts.
- **Measure:** Obtain reproducible time and hardware events where available.
- **Diagnose:** Separate observation, model, inference, and alternative.
- **Partition:** Compare software, hardware, and coupled alternatives.
- **Verify:** Define correctness, interface behavior, and a decisive next test.



Three Sessions — 150 Minutes

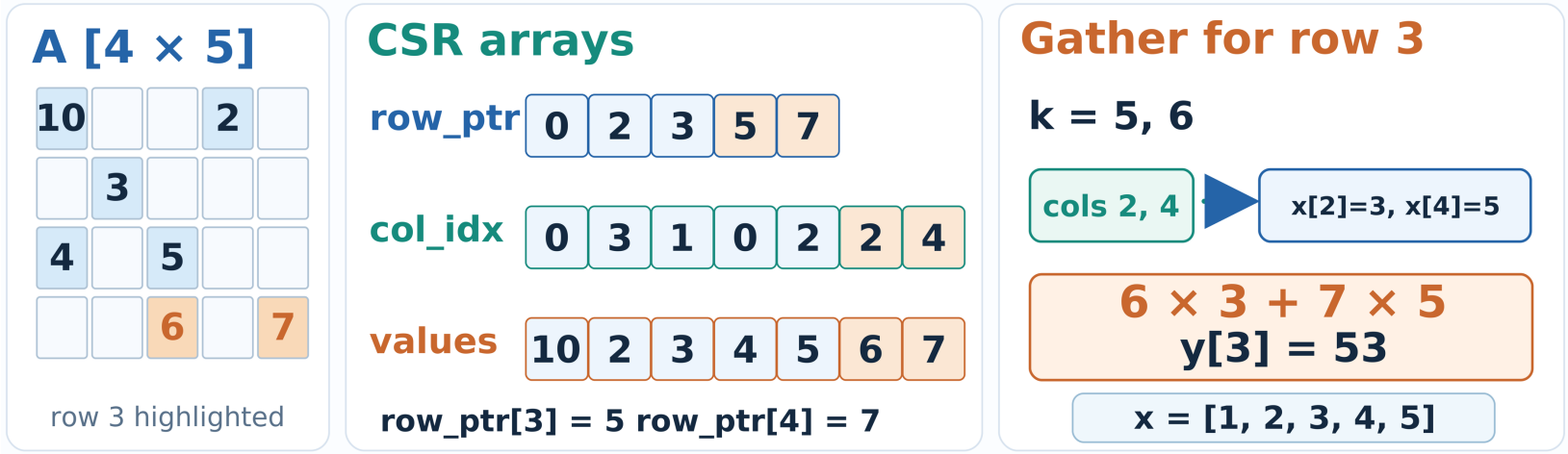
- **Session 1:** Week 3 review (12 min), trace lab (28), debrief (10).
- **Output A:** Functional contract, footprint, 24-access trace, hypothesis.
- **Session 2:** Week 5 review (12 min), profiling lab (28), debrief (10).
- **Output B:** Two correct runs, performance events, diagnosis, limitations.
- **Session 3:** Week 6 theory (15 min), design lab (27), wrap-up (8).
- **Output C:** Partition decision, end-to-end model, verification plan.



Common Workload: CSR SpMV

CSR: compressed sparse row, which is a highly-used sparse format for SpMV

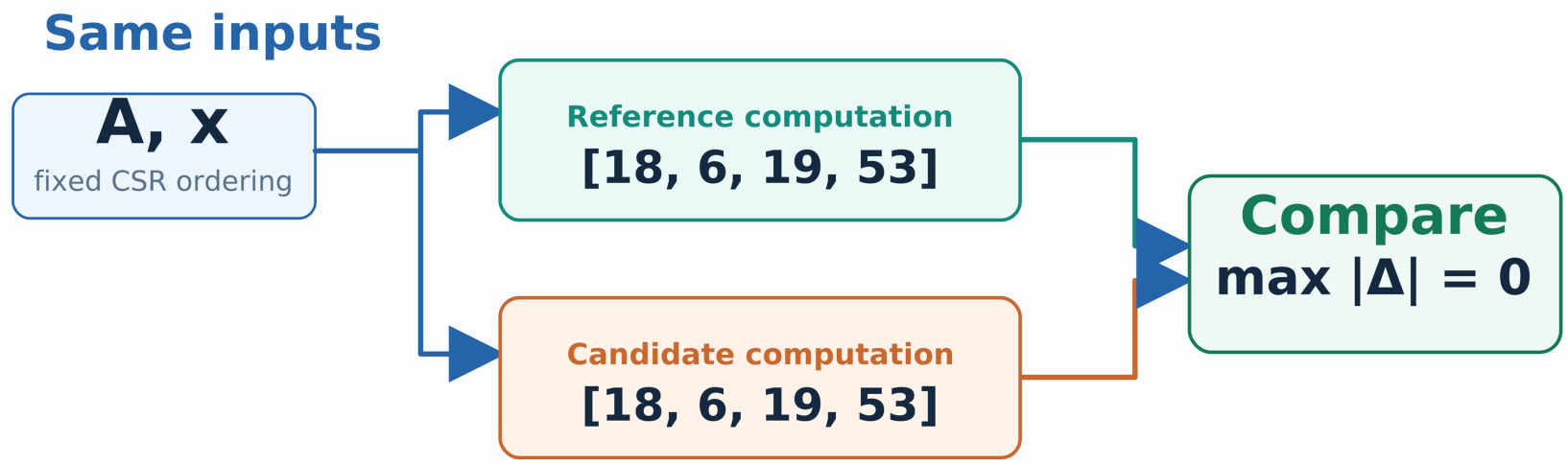
- **Compute:** y[r] = sum over k in row r of val[k] × x[col[k]].
- **CSR:** row_ptr marks row bounds; col_idx and val store nonzeros.
- **Access:** val and col_idx stream; x is indexed indirectly.
- **Output:** One y value per row; preserve the accumulation semantics.
- **Goal:** Study movement and locality before deciding on hardware.



The final entry of row_ptr represents the total #(nonzero entries) in the matrix

Week 3: Freeze a Functional Contract

- **Input:** Matrix dimensions, nonzeros/row, values, x, and ordering.
- **Output:** One double-precision result per matrix row.
- **Oracle:** Compare to a reference result before profiling any variant.
- **Semantics:** Define duplicate columns, zeros, overflow, and sum order.
- **Unit:** One complete SpMV kernel invocation; report rows and nnz.
- **Boundary:** Identify setup, timed kernel, validation, and teardown.



Illustrative example: verify candidate results before timing.

CSR Data Inventory and Footprint

- **row_ptr:** (R + 1) entries; 8 B each on a 64-bit size_t host.
- **col_idx:** NNZ entries × 4 B; a forward stream.
- **values:** NNZ entries × 8 B; a forward stream.
- **x:** C entries × 8 B; indirect, potentially reusable.
- **y:** R entries × 8 B; one output per row.
- **Record:** Actual types, alignment, metadata, and cache boundary.

```
sprasity = 1 - stored_entries/(RxC) ~ 0.99997
```

R = 65,536 C = 262,144 stored entries = 524,288

64-bit size_t | 32-bit uint32_t | FP64 double

Array	Elements × B	MiB
row_ptr	65,537 × 8	**≈ 0.50**
col_idx	524,288 × 4	**2.00**
values	524,288 × 8	**4.00**
x	262,144 × 8	**2.00**
y	65,536 × 8	**0.50**

Five principal arrays
≈ 9.00 MiB

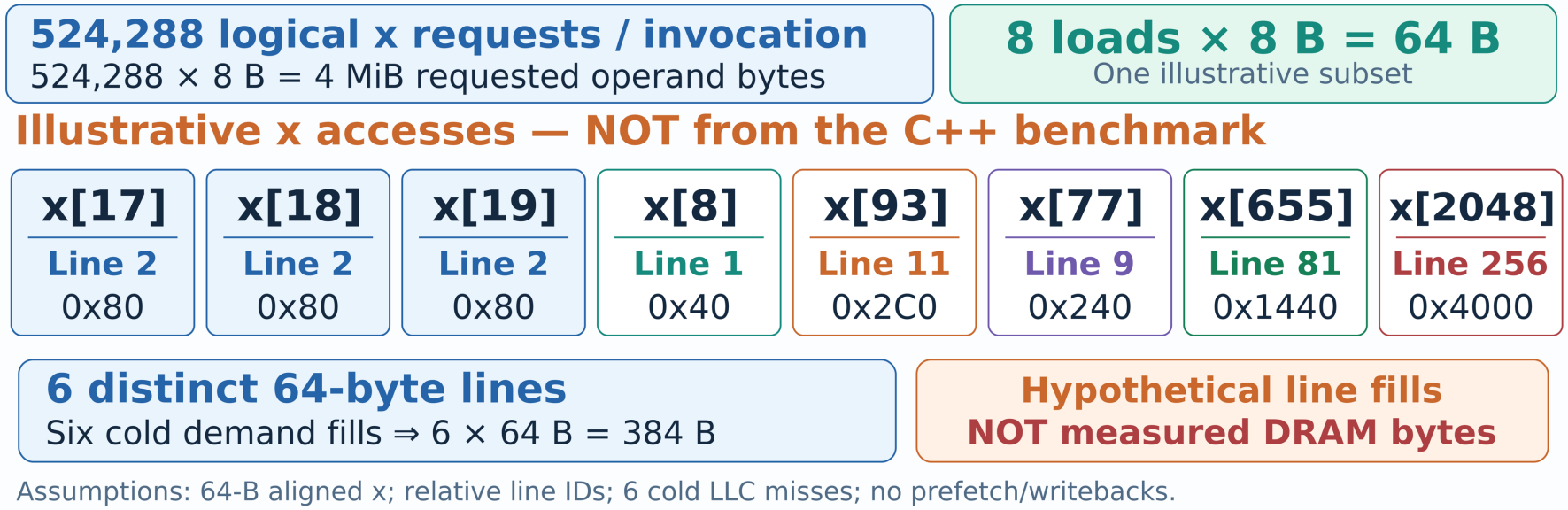
+ expected [R] (oracle)
+ 0.50 MiB

Total allocated arrays
≈ 9.50 MiB

Excludes vector-object/allocator overhead; row_ptr adds 8 B beyond the rounded 9.00 MiB.

Logical Demand Is Not DRAM Traffic

- **Demand:** At 8 NNZ/row, each row requests 8 x elements.
- **Logical x bytes:** 8 B × NNZ per invocation = 4 MiB here.
- **Reuse:** Cache lines can satisfy more than one operand request.
- **Physical bytes:** Count transfers at a named cache/DRAM boundary.
- **Extra effects:** Line fills, evictions, write allocation, and prefetch.
- **Rule:** Never substitute logical operands for measured DRAM traffic.

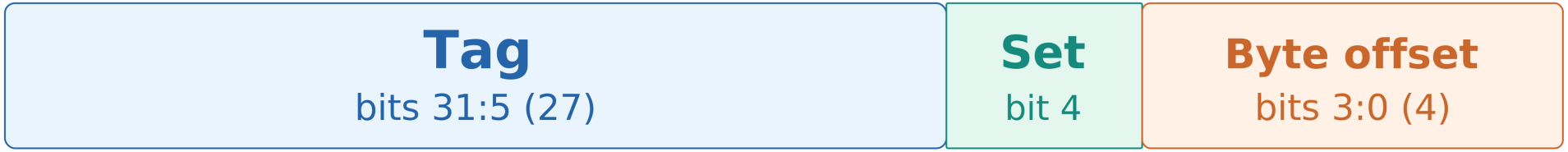


Trace: Lines, Pages, and Sets

- **Line ID:** floor(byte address / cache-line bytes).
- **Page ID:** floor(byte address / page bytes).
- **Cache set:** line ID mod set count for a simple modulo-index model.
- **Reuse:** Record repeated lines, cold misses, and victims.
- **Dependencies:** Mark independent addresses vs pointer-dependent ones.
- **Caution:** Real DRAM row/bank mapping needs platform documentation.

Simple physically addressed cache model

32-bit addresses • 16-B cache lines • 2 sets • 4-KiB pages

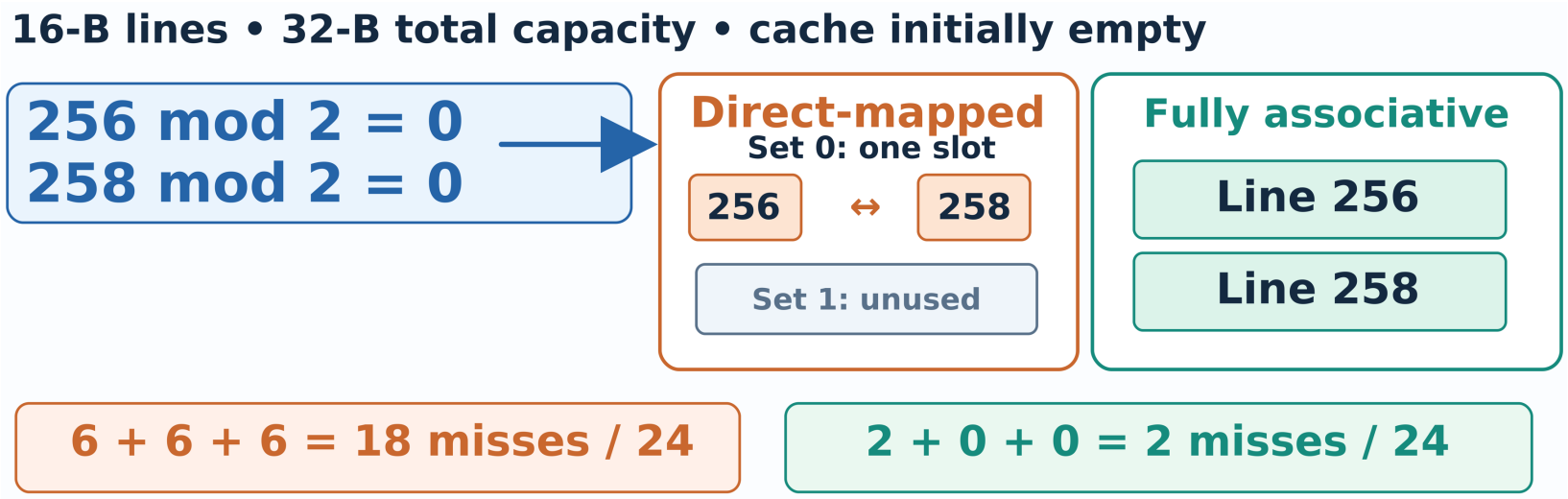


Schematic fields, not drawn to bit scale

Address	Line ID	Page ID	Set	Offset
0x1000	256	1	**0**	0
0x1004	256	1	**0**	4
0x1020	258	1	**0**	0
0x1024	258	1	**0**	4

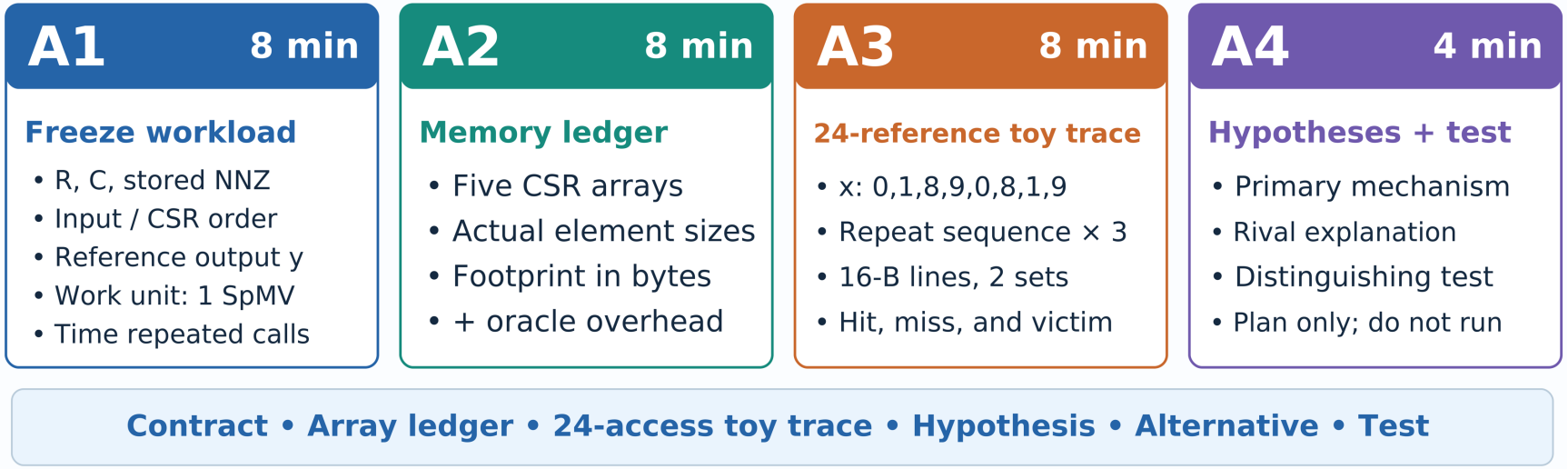
Toy Cache: A Controlled Conflict

- **Assume:** 16-B lines and 32-B total cache capacity.
- **Trace:** [0,1,8,9,0,8,1,9] repeated three times.
- **Addresses:** x begins at 0x1000; 4 B per toy element.
- **Mapping:** Lines 256 and 258 collide in a 2-set direct map.
- **Prediction:** Direct-map 18 misses; 2-way fully-assoc 2.
- **Scope:** 24 simulated references, not measured CPU cache misses.



Lab A — Workload and Trace (28 min)

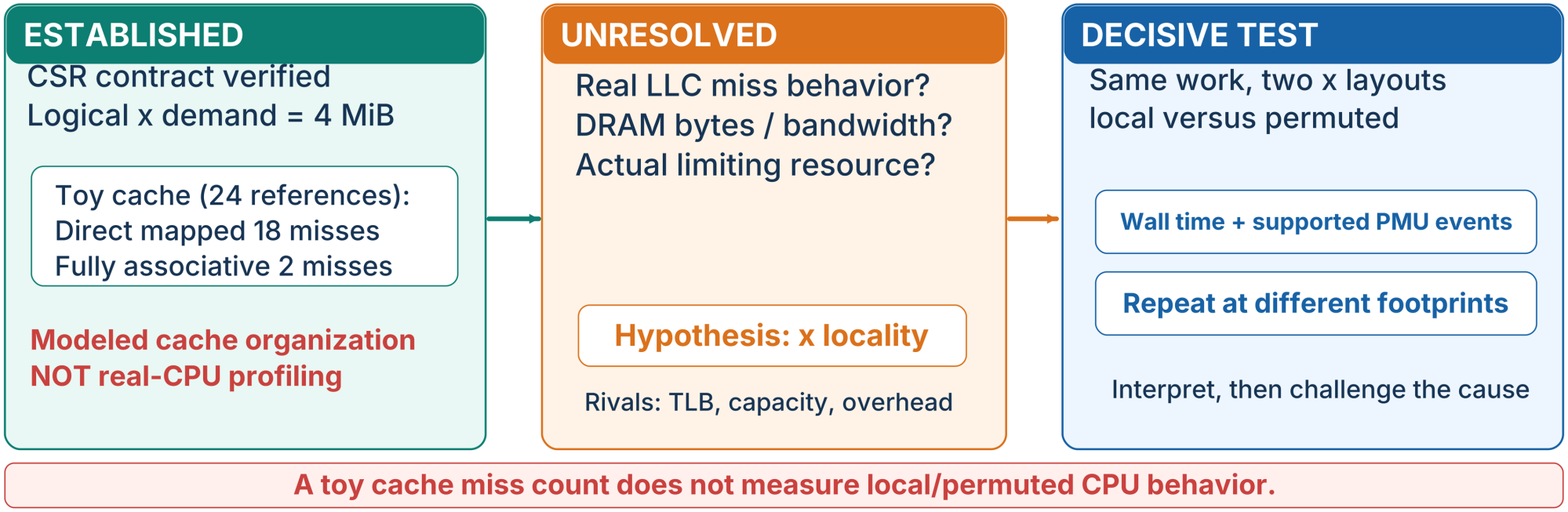
- **A1 • 8 min:** Freeze dimensions, oracle, work unit, timed region.
- **A2 • 8 min:** Complete the five-array footprint ledger.
- **A3 • 8 min:** Trace 24 references; label line, set, hit/miss.
- **A4 • 4 min:** Write one cause, one alternative, and an attack test.
- **Hand in:** Ledger plus annotated trace with assumptions and units.



Primary hypothesis: Cache-set conflicts are causing unnecessary line re-fetches.
Alternative explanation: On a real processor, a slowdown might instead involve insufficient effective cache capacity or address-translation overhead.
Discriminating test: Compare equal-capacity caches with different associativity in the toy model, or later measure equivalent workloads with controlled changes in data layout.

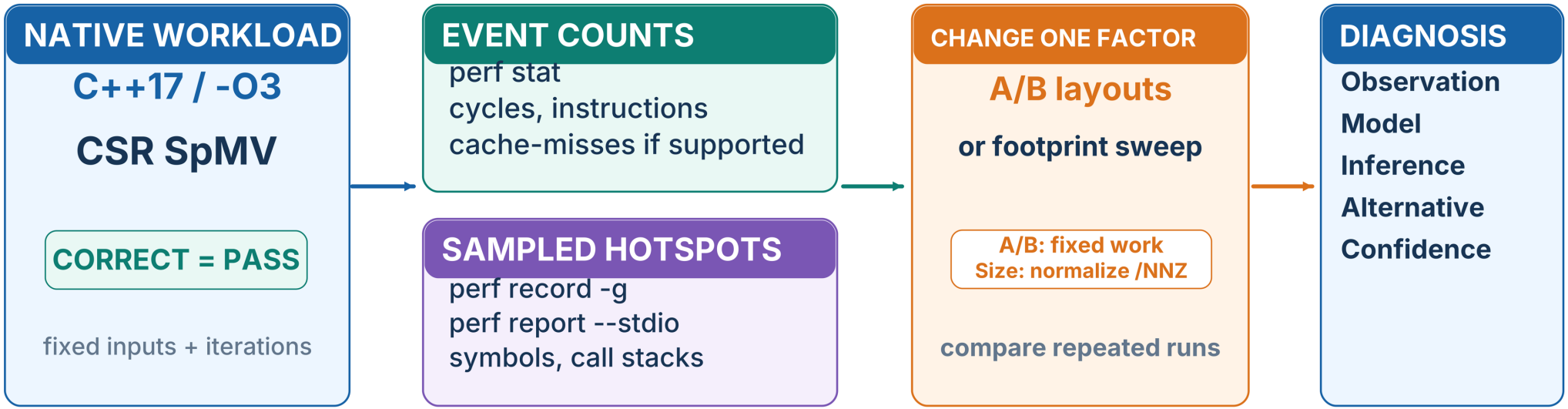
Week 3 Exit: Evidence vs Hypothesis

- **Established:** CSR contract, logical operands, and toy-model behavior.
- **Not established:** Real LLC miss rate or DRAM bandwidth.
- **Primary hypothesis:** Irregular x layout reduces effective locality.
- **Alternative:** Cache capacity, translation, or execution overhead.
- **Test:** Profile two equivalent layouts and vary input footprint.
- **Carry forward:** Every Week 5 claim needs evidence and a falsifier.



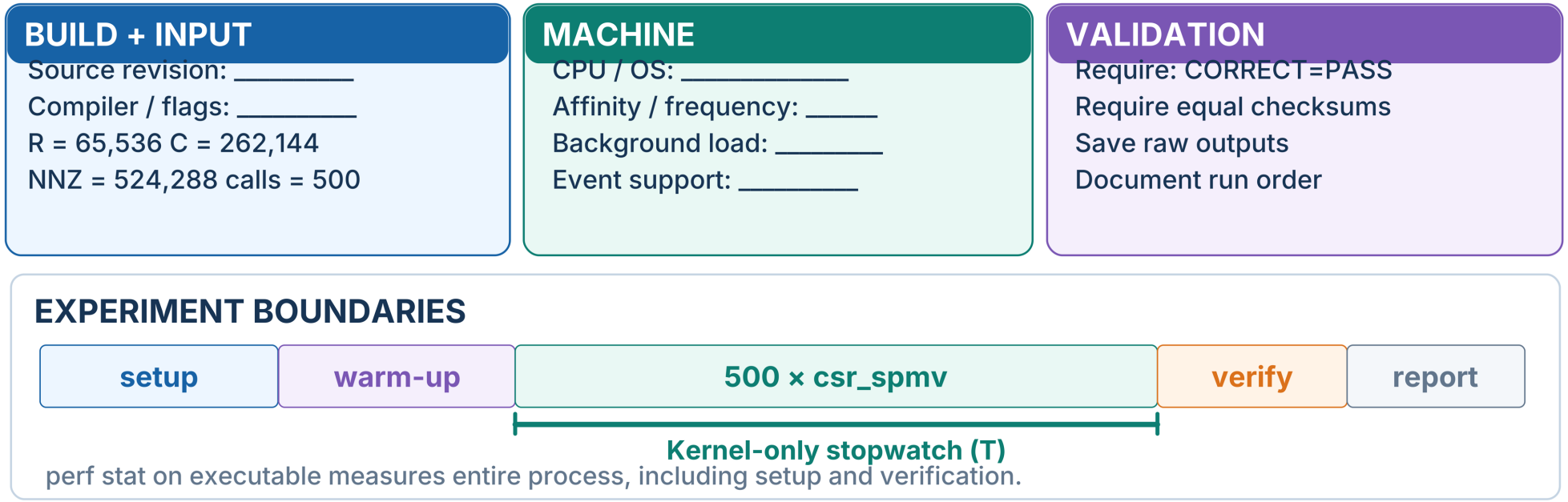
Week 5: Diagnose with Real Evidence

- **Benchmark:** Compile the data-intensive kernel as optimized native code.
- **Measure:** Timing and accessible PMU events with Linux perf.
- **Locate:** Sample hot functions using perf record and perf report.
- **Challenge:** Change one locality or size variable at a time.
- **Explain:** Build a five-layer diagnosis with counterexamples.
- **Extend:** Google gperftools / VTune are optional, not required.



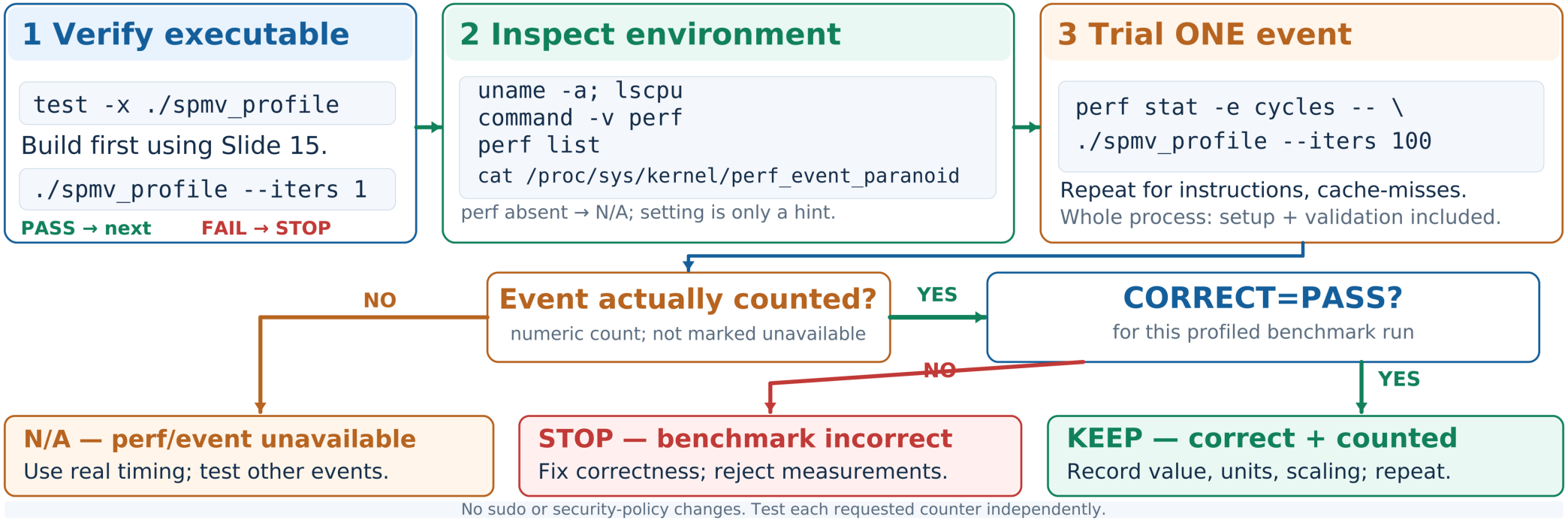
Freeze the Profiling Experiment

- **Build:** Source revision, compiler version, flags, and executable.
- **Data:** R, columns, nonzeros/row, pattern, and iteration count.
- **Host:** CPU model, OS, affinity, frequency policy, background load.
- **Outcome:** CORRECT=PASS and the same checksum for both variants.
- **Scope:** Kernel timer excludes setup; process-level perf does not.
- **Repetition:** Keep raw results, run order, and dispersion.



Linux perf: Check Availability First

- **Environment:** Run on native Linux, not macOS perf commands.
- **Inspect:** uname -a; lscpu; command -v perf; perf list.
- **Access:** Read /proc/sys/kernel/perf_event_paranoid.
- **Counters:** Check events on this specific processor.
- **Fallback:** If PMU blocked, mark unavailable; use timing + trace.
- **Policy:** Do not require sudo or security-setting changes.



Compile and Validate the Baseline

- **Compile:** C++17, -O3, -g, -fno-omit-frame-pointer.
- **Execute:** Two runs with --pattern local and permuted.
- **Fix:** --rows 65536; --nnz-per-row 8; --iters 500.
- **Require:** CORRECT=PASS with identical checksums.
- **Record:** timed_kernel_s, ms_per_iter, and ns_per_nnz.
- **Failure:** Stop before profiling and investigate output mismatch.

BUILD (C++17, optimized, with symbols)

```
g++ -O3 -g -fno-omit-frame-pointer -std=c++17 \
Week6_Profiler_Based_CSR_Benchmark.cpp -o spmv_profile
```

A / contiguous

```
./spmv_profile --pattern local \
--rows 65536 --nnz-per-row 8 \
--iters 500
```

CORRECT=PASS; keep checksum and timings

B / shuffled

```
./spmv_profile --pattern permuted \
--rows 65536 --nnz-per-row 8 \
--iters 500
```

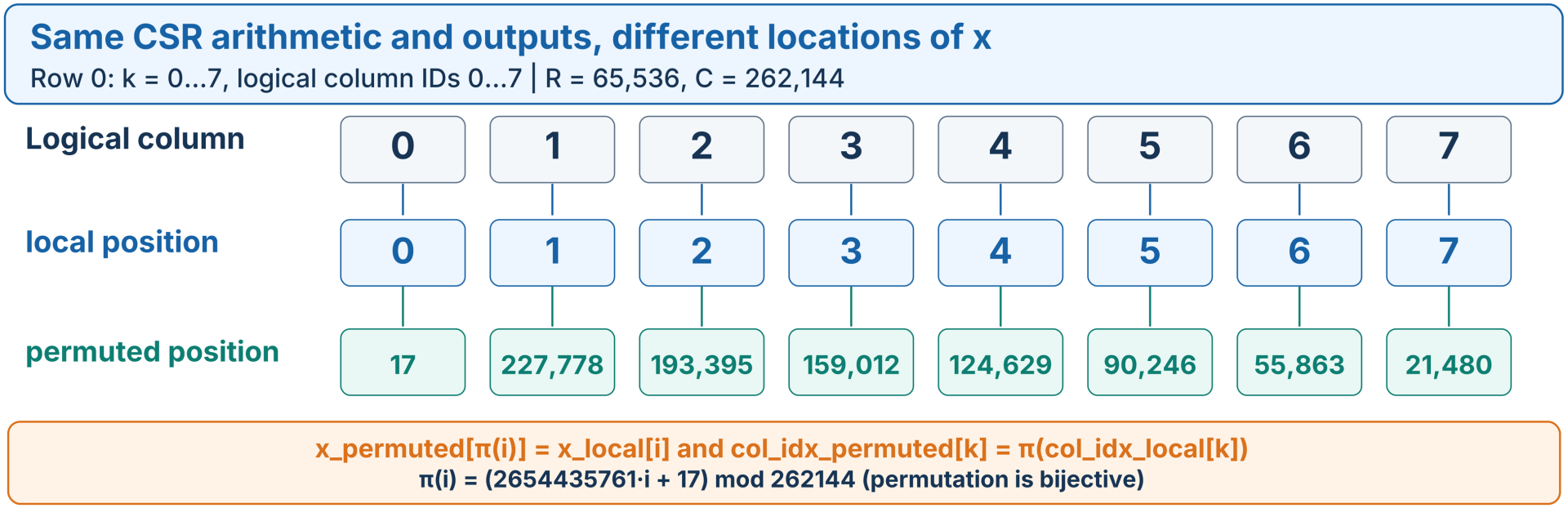
CORRECT=PASS; keep checksum and timings

The checksums must match across both layouts.

If a correctness check fails, do not compare performance.

Why Compare Local and Permuted?

- **Same work:** Matrix dimensions, nonzeros, operations, output.
- **Local:** Consecutive logical x elements are colocated in memory.
- **Permuted:** Both x storage and column indices are remapped.
- **Controlled:** Only operand address order/layout is intended to differ.
- **Question:** Do time and cache-related events respond to locality?
- **Caution:** Neither layout is guaranteed faster on every host.



Count Events with perf stat

- **Command:** perf stat -r 3 -e cycles,instructions,cache-misses.
- **Target:** Add -- then the full spmv_profile command.
- **Compare:** Local and permuted with identical dimensions/iters.
- **Capture:** Instructions, cycles, misses, and elapsed time.
- **Qualify:** Event support, scaling, counter time-running, and errors.
- **Warning:** Generic cache-misses is not a direct DRAM-byte count.

```
perf stat -r 3 -e cycles,instructions,cache-misses -- ./spmv_profile [arguments]
```

local

Elapsed time	_____	s
Cycles	_____	events
Instructions	_____	events
Cache misses	_____	events

permuted

Elapsed time	_____	s
Cycles	_____	events
Instructions	_____	events
Cache misses	_____	events

IPC = instructions / cycles

Compare event counts and times separately.

cache-misses ≠ DRAM traffic

Event semantics, permissions and scaling vary.

Locate Hotspots with perf record

- **Sample:** perf record -g -- ./spmv_profile [arguments].
- **Inspect:** perf report --stdio to view expensive symbols.
- **Expect:** The compiled csr_spmv kernel should be observable.
- **Interpret:** Samples indicate where cycles accrue, not bytes moved.
- **Limit:** Frequency, unwind quality, and symbols affect attribution.
- **Fallback:** Keep counter and timing evidence if sampling is blocked.

SAMPLING

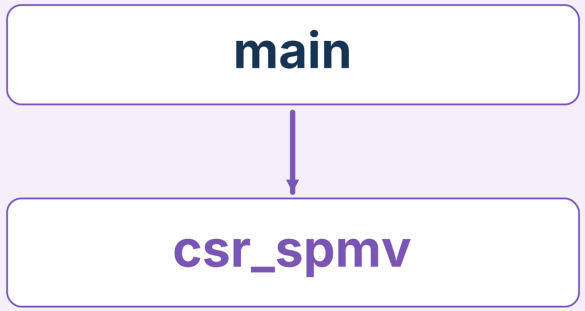
```
perf record -e cycles -g -- \
./spmv_profile --pattern permuted \
--rows 65536 --nnz-per-row 8 \
--iters 500
```

↓

```
perf report --stdio
```

Call stacks & hot symbols
If event access permits

CALL PATH (SCHEMATIC)



Function attribution depends on symbols and stack unwinding; percentages omitted

WHAT IT MEASURES

Sampled execution events (configured here: cycles)

NOT bytes transferred
NOT exact time per function
NOT a DRAM bandwidth counter

If sampling is unavailable, retain valid time/counter evidence and mark sampling N/A.

Normalize without Over-Interpreting

- **IPC:** Retired instructions divided by core cycles, if supported.
- **Kernel time:** Report ms/invocation and ns/nonzero.
- **Proxy:** Cache-miss events per total timed nonzeros.
- **Caveat:** perf counts setup and validation too; ensure amortization.
- **Repeat:** Record median/spread and raw measurements.
- **Do not infer:** DRAM saturation solely from generic cache events.

T = timed_kernel_s (seconds) N = iterations K = NNZ per invocation

KERNEL TIME (T)

ms / invocation = $1000\ T / N$

ns / nonzero = $10^9\ T / (N\ K)$

Default: N=500, K=524,288

PROCESS PMU EVENTS

IPC = instructions / cycles

misses / nonzero = $M / (N\ K)$

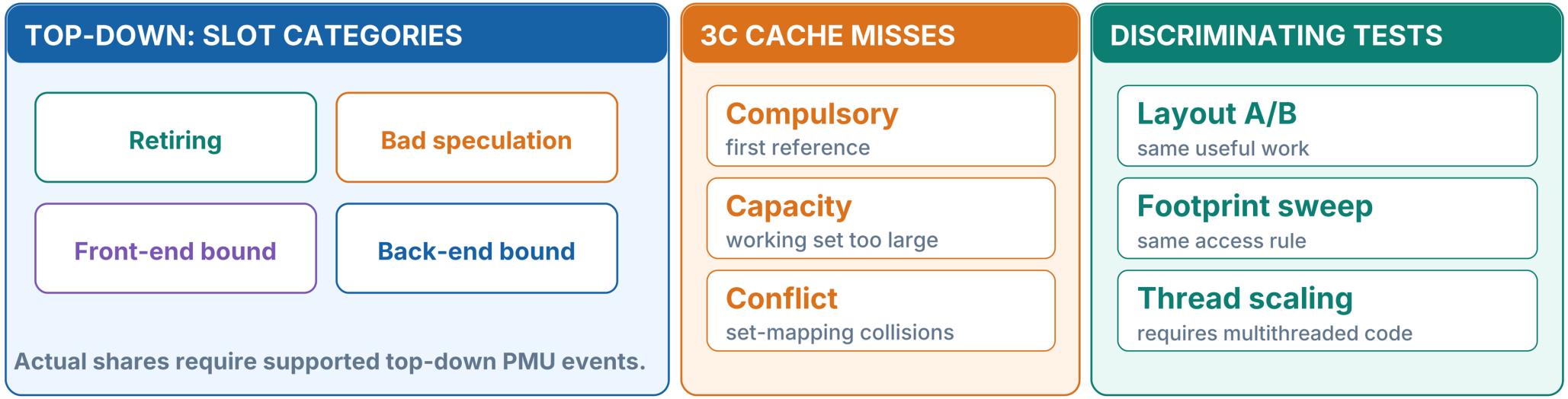
M = process-level cache-miss count

Kernel stopwatch excludes setup / verification; process perf counters generally include them.
Amortize overhead, preserve raw events, and avoid claims of DRAM saturation.

TMAM, 3C, and Competing Causes

- **TMAM:** Retiring, bad speculation, front-end, back-end categories.
- **Need:** Architecture-specific top-down events for true TMAM shares.
- **3C:** Compulsory, capacity, conflict: explain with controlled models.
- **Attack:** Change layout at fixed work; sweep footprint or threads.
- **Alternative:** Core overhead, TLB misses, or frequency variability.
- **Confidence:** Mark each conclusion measured, inferred, or unknown.

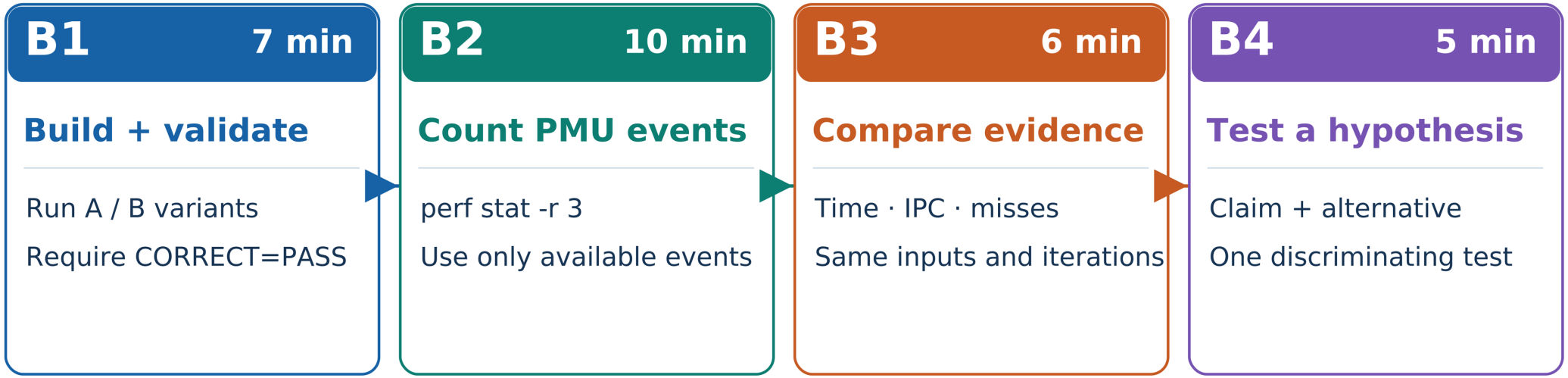
Benchmark constraint: `rows` must be a power of two; `C = 4R`. Use valid sizes for footprint sweeps and normalize results by total work.



No TMAM percentages or DRAM bottleneck claims can be derived from generic cache misses alone.
Label conclusions MEASURED / INFERRED / UNKNOWN.

Lab B — Real Profiling (28 min)

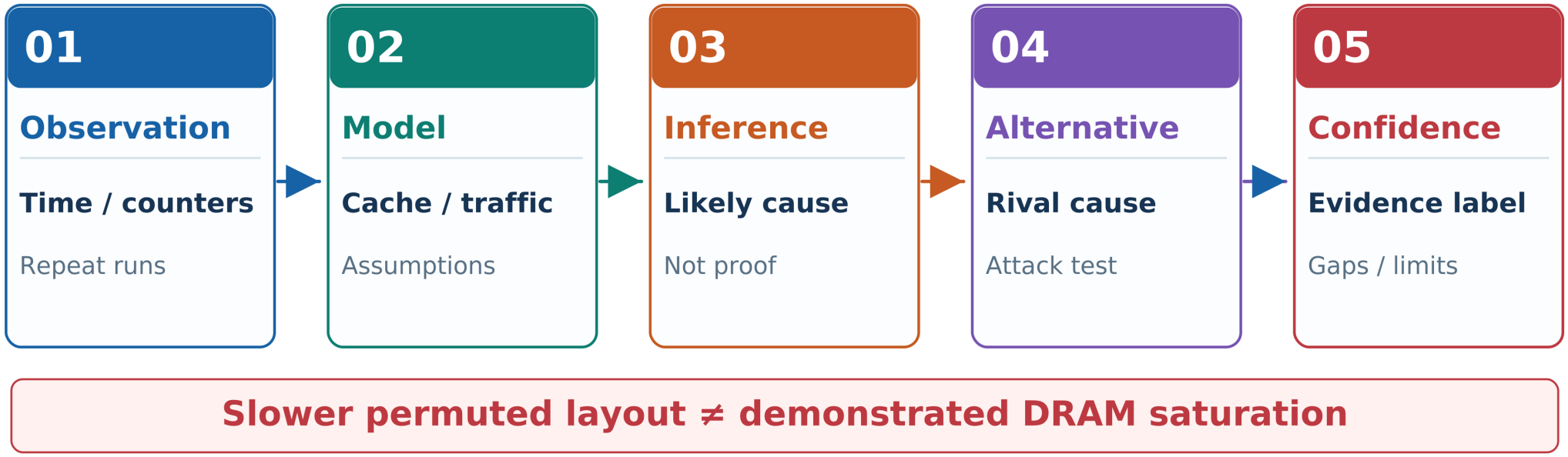
- **B1 • 7 min:** Build both variants; verify matching outputs.
- **B2 • 10 min:** Run perf stat -r 3 for both patterns.
- **B3 • 6 min:** Draft a baseline table; compare IPC and miss events.
- **B4 • 5 min:** State leading hypothesis + one disproof test.
- **Stretch:** Size sweep + mechanism plot; perf record if accessible.
- **Fallback:** If PMU denied, mark N/A; keep real time evidence.



Checkpoint B
Two correct runs • baseline table • diagnosis • PMU limits

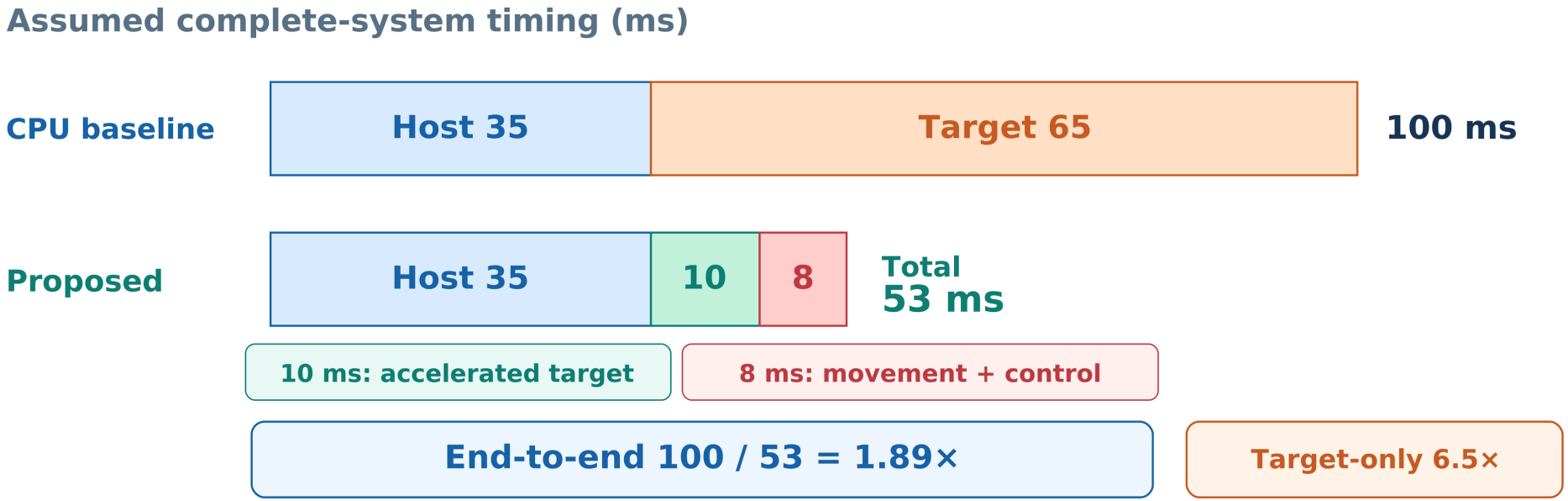
Five-Layer Bottleneck Diagnosis

- **1. Observation:** Quantified times/events and inputs.
- **2. Model:** Cache/traffic prediction or achievable bound.
- **3. Inference:** Best-supported limiting mechanism.
- **4. Alternative:** A competing cause plus a test separating them.
- **5. Confidence:** List unsupported counters and remaining uncertainty.
- **Rule:** A slower permuted pattern alone does not prove DRAM bound.



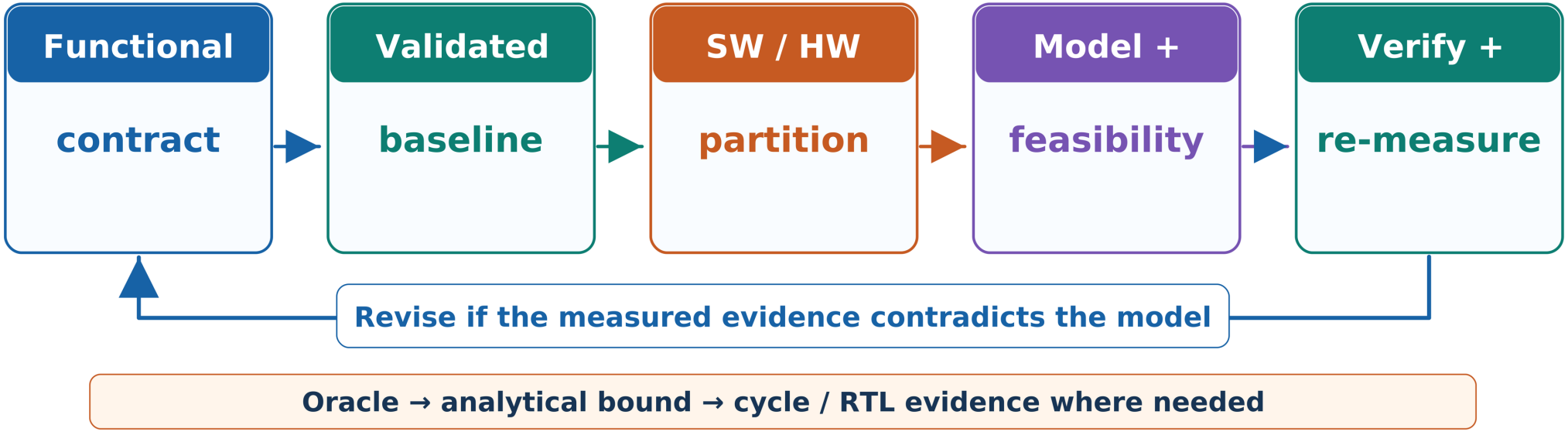
Amdahl: Include the Whole System

- **Assumed baseline:** 100 ms = 35 ms host + 65 ms target.
- **Assumed proposal:** Target 10 ms; added movement/control 8 ms.
- **New total:** 35 + 10 + 8 = 53 ms.
- **System gain:** 100 / 53 = 1.89×, not 6.5×.
- **Attack:** How does a software-only data layout change compare?
- **Constraint:** Costs and target fractions must share a fair boundary.



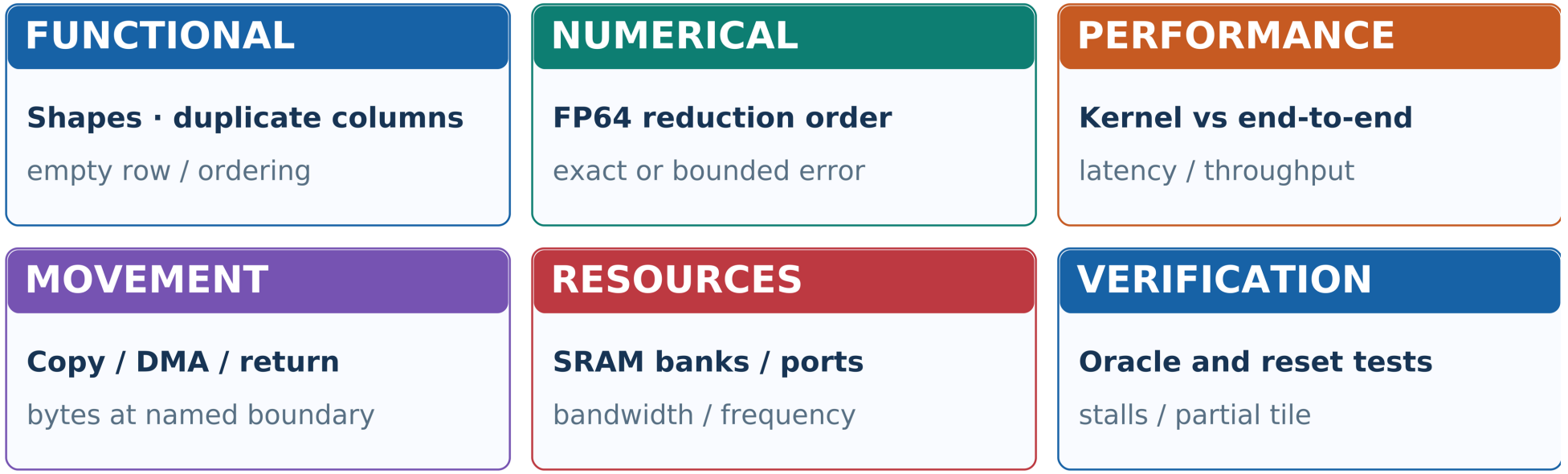
Week 6: Co-Design Follows Evidence

- **Specify:** Define functional, time, movement, resource constraints.
- **Partition:** Assign host, DMA, buffers, datapath, and output.
- **Estimate:** Test transfer, compute, buffering, and launch cost.
- **Compare:** Reject dominated or infeasible implementations early.
- **Refine:** Golden model → analytical model → cycles → RTL.
- **Revisit:** New measurements can invalidate the assumed bottleneck.



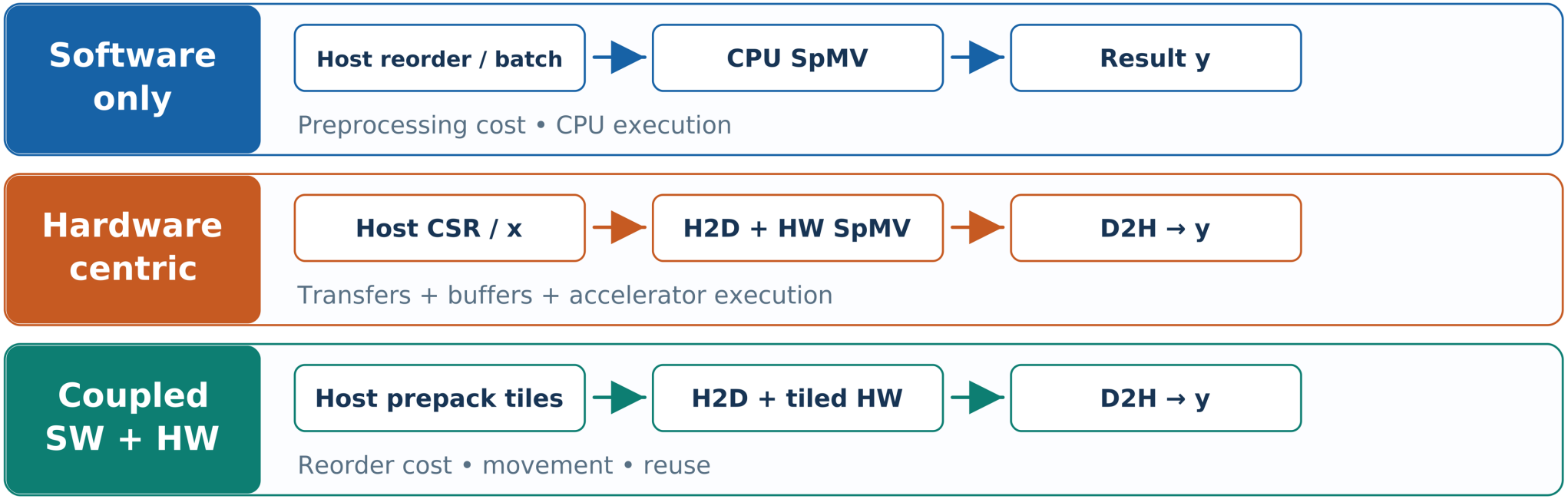
Specify the Complete Design Contract

- **Functional:** Legal shapes, duplicate indices, reset, output ordering.
- **Numerical:** Sum order, exactness/tolerance, rounding and overflow.
- **Performance:** Kernel rate AND end-to-end latency/throughput.
- **Movement:** Host copies, bursts, alignment, read/write bytes.
- **Resource:** On-chip memory, ports, bandwidth, clock, power.
- **Verification:** Oracle vectors, protocol invariants, and edge cases.



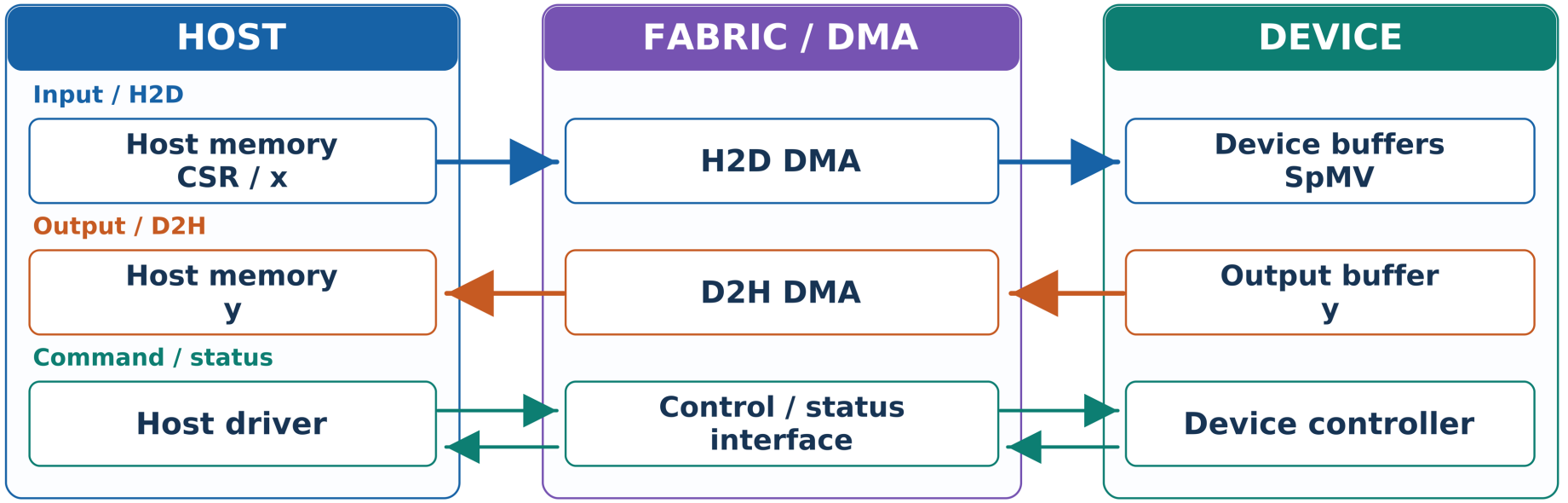
Three Distinct Interventions

- **Software-only:** Improve ordering, batching, packing, or thread schedule.
- **Hardware-centric:** Build a stream processing datapath and buffers.
- **Coupled:** Prepack/reorder input in software and stream hardware tiles.
- **Compare:** Affected bottleneck, expected end-to-end benefit.
- **Include:** Preprocessing, transfers, extra memory, and verification.
- **Reject:** Any choice that breaks correctness or the resource limit.



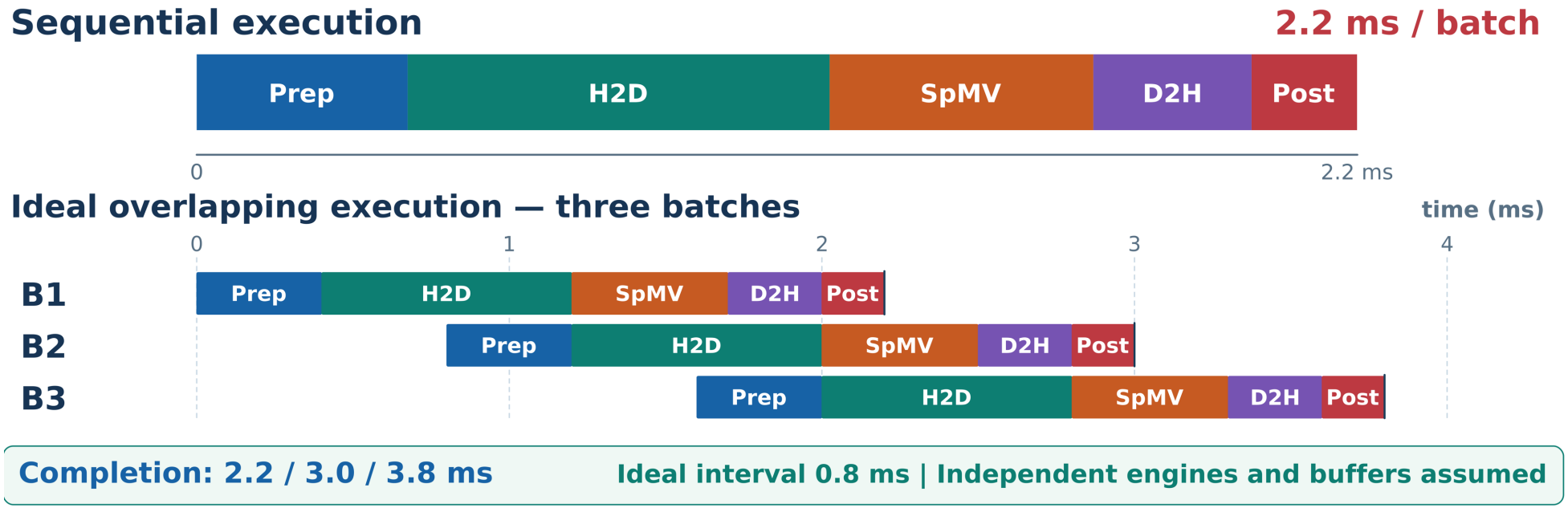
Model the Full Host–Hardware Path

- **Host:** Allocate, validate, pack/reorder, and submit work.
- **Transfer in:** Move descriptors, row pointers, values, indices, x.
- **Local storage:** Scratchpad, banks, tags, and buffer allocation.
- **Compute:** Fetch indexed operands; accumulate and resolve conflicts.
- **Transfer out:** Return y and completion/error status.
- **Ask:** Where does the profiler suggest the limiting boundary?



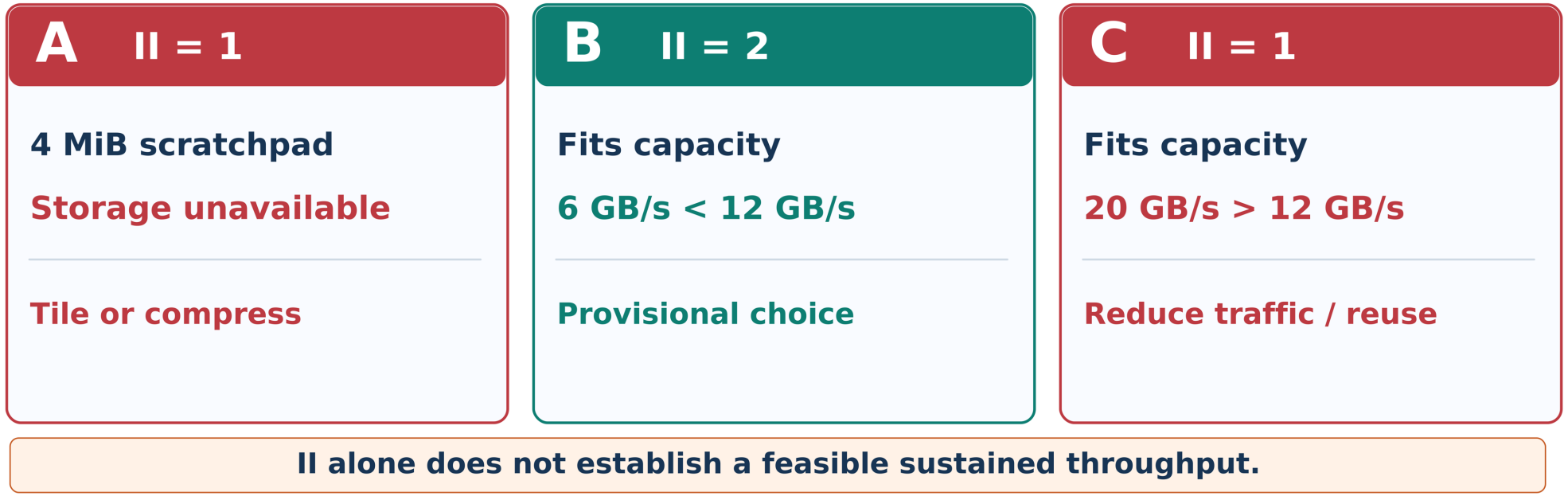
Serial vs Overlapped Execution

- **Assumed stages:** Prep .4; in .8; compute .5; out .3; post .2 ms.
- **Serialized:** First-batch = 2.2 ms under this stage model.
- **Ideal stream:** Long-run interval ≥ max(stage times) = 0.8 ms.
- **Needed:** Independent engines, buffers, concurrency, no stalls.
- **Reality:** Shared DMA/ports or dependencies may prevent overlap.
- **Do not claim:** Double buffering alone guarantees 0.8 ms.



Resource and Bandwidth Gates

- **A:** II=1 but needs unavailable 4 MiB on-chip storage.
- **B:** II=2, fits storage, and needs 6 GB/s sustained.
- **C:** II=1 but demands 20 GB/s on a 12 GB/s interface.
- **Screen:** B is the only provisionally feasible candidate.
- **Rescue A:** Tile or compress state within available capacity.
- **Rescue C:** Increase reuse, reduce bytes, or lower demanded rate.



Ready/Valid and Backpressure

- **Handshake:** Transfer occurs at clock edge iff valid and ready.
- **Stall:** With valid=1 and ready=0, hold item and valid stable.
- **Capacity:** A full FIFO must prevent overflow; upstream acceptance may continue if a simultaneous dequeue safely frees capacity.
- **Ordering:** Define tags or strict in-order completion.
- **Robustness:** State reset, error, timeout, and partial-batch rules.
- **Assert:** No accepted item is dropped or duplicated.

Sample VALID and READY together at each rising clock edge.

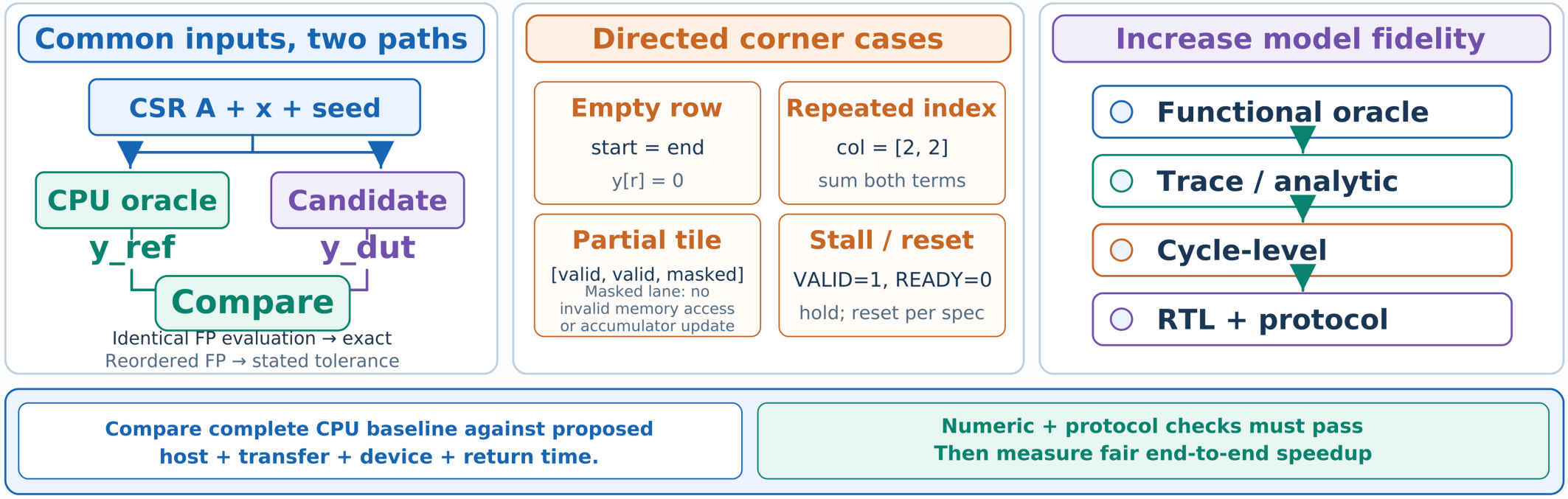
	E1	E2	E3	E4	E5	E6	E7
VALID	0	1	1	1	1	1	0
READY	1	0	0	1	0	1	1
DATA	—	A	A	A	B	B	—
EDGE	—	STALL	STALL	TAKE A	STALL	TAKE B	—

VALID = 1 and READY = 0 ⇒ hold DATA and VALID stable

Transfers occur at E4 (A) and E6 (B); no transfer on stalled edges.

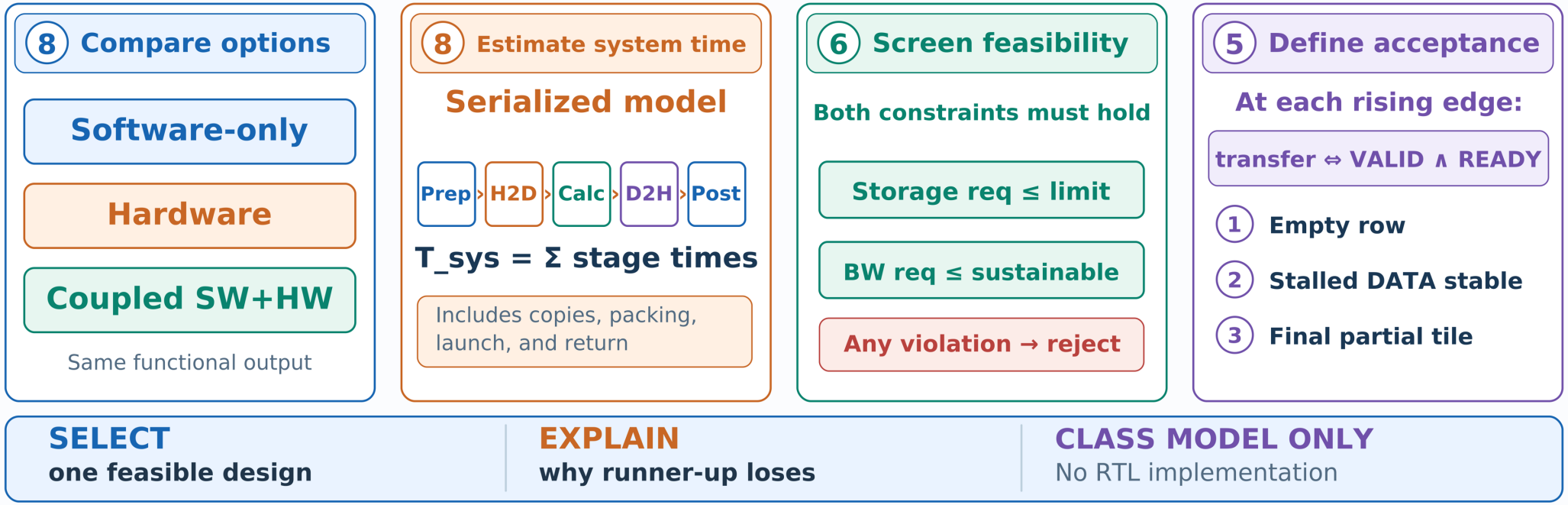
Verify Before Claiming Speedup

- **Golden model:** Compare output for a fixed workload and seed.
- **Directed cases:** Empty rows, duplicate indices, final partial tile.
- **Numeric policy:** Account for floating-point sum reordering.
- **Protocol cases:** Backpressure, queue full/empty, reset mid-flight.
- **Progression:** Functional → trace/analytical → cycle → RTL.
- **Closure:** Hardware results must reconnect to end-to-end CPU baseline.



Lab C — Defend a Partition (27 min)

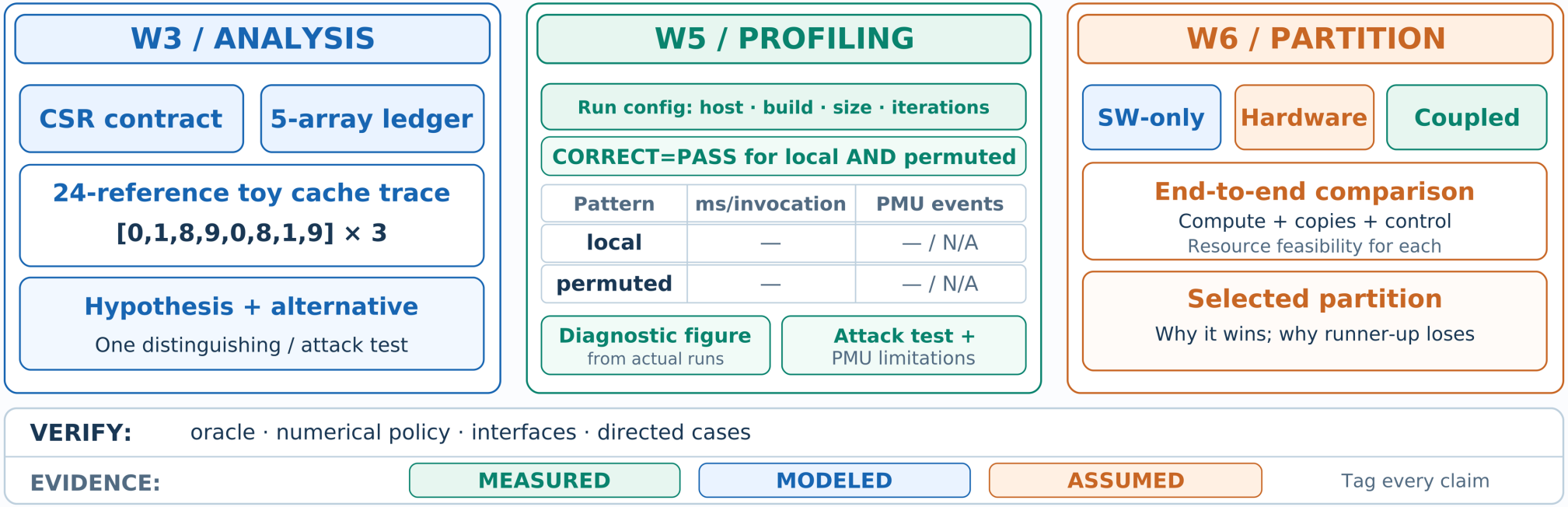
- **C1 • 8 min:** Compare SW-only, HW, and coupled alternatives.
- **C2 • 8 min:** Model total time including copies and buffering.
- **C3 • 6 min:** Reject infeasible designs under one hard resource.
- **C4 • 5 min:** Define ready/valid semantics and 3 directed tests.
- **Deliver:** One selected design and why the runner-up loses.
- **Caution:** Class design remains modeled, not implemented RTL.



One Integrated Checkpoint Submission

- **Week 3:** Contract, memory ledger, 24-access toy trace, hypothesis.
- **Week 5:** Reproducible baseline table, run config, PASS.
- **Week 5:** One diagnostic figure, attack test, PMU limitations.
- **Week 6:** Three alternatives, end-to-end comparison, chosen partition.
- **Verification:** Oracle/numerics, interfaces, and directed edge cases.
- **Integrity:** Label each result measured, modeled, or assumed.

Submission: One integrated guided checkpoint covering Weeks 3, 5, and 6. Due: [announced date/time]. Unavailable PMU events may be reported as N/A with valid timing evidence.



Decision Gate and Week 7 Bridge

- **Keep:** Only a correct, reproducible baseline.
- **Explain:** Most likely bottleneck and at least one rival cause.
- **Quantify:** System—not kernel-only—benefit and movement cost.
- **Choose:** Feasible SW/HW boundary with an explicit resource budget.
- **Plan:** Next experiment to invalidate the weakest assumption.
- **Week 7:** Explore dataflow, buffering, tiling, and microarchitecture.

